Analisi Tecnica del Firmware

Progetto *Vera-Module-RBL-C64*

Analisi del progetto a cura di RetroBit Lab

26 ottobre 2026

Indice

1 Introduzione **1**

2 Architettura del Sistema **1**
2.1 Componenti Chiave 1
2.2 Mappa I/O 1
2.3 Ambiente di build PlatformIO 2
2.4 Interfacciamento Bus 2
2.5 Ripartizione dei core 3

3 Mapping dei Pin (ESP32-S3FN8, QFN56) **3**
3.1 Pin di strapping — comportamento al boot con il C64 collegato 4
3.2 Quarzo oscillatore 40 MHz (pin 53–54) 5

4 Analisi delle Tempistiche **5**
4.1 Sequenza di un ciclo 6
4.2 Percorso critico 6
4.3 Accesso diretto ai registri GPIO 7
4.4 Diagramma temporale di una lettura GeoRAM 7

5 Frammento di Codice Sorgente **7**

6 Diagramma di Flusso del Firmware **8**

7 Verifiche hardware **9**

8 Conclusione **10**

1 Introduzione

Questo documento presenta un'analisi tecnica del firmware ESP32 per il progetto *Vera-Module-RBL-C64*. Il firmware, eseguito su un ESP32-S3, si interfaccia con il bus del processore 6510 attraverso la **porta di espansione del Commodore 64**. Espone i registri della scheda video VERA (FPGA) e una **GeoRAM da 256 KB** emulata nella RAM interna dell'ESP32.

Il progetto deriva dalla versione Atari (*Vera-Module-RBL-XE*). Lo schema è quello di RBL-XE nella revisione del 2026-10-26, con il foglio del connettore adattato alla porta di espansione del C64; la piedinatura dell'ESP32-S3 è la stessa di RBL-XE, tranne GPIO 41/42 che diventano gli ingressi `/IO1`/`/IO2` (`PIN-MAPPING.md` §9). Il PCB è ancora quello Atari. Anche il firmware riprende l'architettura di RBL-XE (loop del bus sul Core 1 a interruzioni mascherate, percorso critico in assembly, tabelle di decodifica, log senza lock), adattata al bus del C64. L'analisi si concentra sulla fattibilità temporale dell'interazione fra la MCU e la CPU e sulla struttura del loop del bus. **Nulla è stato provato su hardware.**

2 Architettura del Sistema

2.1 Componenti Chiave

- **CPU 6510 (MOS):** il processore del C64, a **0.985 MHz** (PAL) o **1.023 MHz** (NTSC). PHI2 alto dura ≈490 ns.
- **MCU: ESP32-S3FN8** (QFN56, 7×7 mm) — dual-core Xtensa LX7 @ 240 MHz, 512 KB SRAM, 8 MB flash in-package Quad SPI, 45 GPIO.
- **FPGA VERA:** iCE40UP5K con il firmware VERA; risponde ai registri $DF00–$DF1F quando l'ESP32 asserisce `DEV_SEL_N`.

2.2 Mappa I/O

La scheda occupa le due aree I/O della porta di espansione, `IO1` ($DE00–$DEFF) e `IO2` ($DF00–$DFFF). Sul C64 queste aree non hanno memoria interna: non serve disabilitare nulla sulla scheda madre, basta pilotare il bus dati.

Indirizzo	Accesso	Funzione	Chi risponde
$DE00–$DEFF	R/W	Finestra GeoRAM (pagina da 256 B)	ESP32, `georam_256k[]`
$DF00–$DF1F	R/W	Registri VERA	FPGA, `DEV_SEL_N` asserito
$DFFD	W	Latch selezione VERA	ESP32 (snoop)
$DFFE	W	GeoRAM, registro pagina (0–63)	ESP32 (snoop)
$DFFF	W	GeoRAM, registro blocco (0–15)	ESP32 (snoop)

La VERA risponde **solo se selezionata**: un valore $\neq 0$ scritto in $DFFD la seleziona, `0` la deseleziona. La GeoRAM (finestra e registri $DFFE/$DFFF) è **sempre attiva**, come nella cartuccia originale. Il latch risponde sempre.

La GeoRAM è organizzata in 16 blocchi da 16 KB, ognuno di 64 pagine da 256 byte:

$$\text{offset} = \text{blocco} \times 16384 + \text{pagina} \times 256 + (\text{addr} \,\&\, \$FF)$$

I registri sono a sola scrittura, si azzerano all'accensione e sopravvivono al reset del C64, come nell'hardware originale. Sono decodificati esattamente a $DFFE/$DFFF, senza i mirror $DF80–$DFFD di alcune GeoRAM: per questo il latch a $DFFD non va in conflitto.

2.3 Ambiente di build PlatformIO

Il file `platformio.ini` definisce un solo ambiente, `esp32s3fn8`. Opzioni di compilazione (`-D`):

Macro	Default	Effetto
`CARD_SEL_LATCH`	`0xFD`	offset in `IO2` del latch di selezione ($DFFD)
`IO_QUALIFY_NS`	200	attesa massima di `/IO1`/`/IO2` dopo il fronte di salita di PHI2
`BUS_WR_SAMPLE_NS`	300	ritardo dalla salita di PHI2 al campionamento del dato in scrittura
`VERA_TRACE_REGS`	0	log di ogni accesso ai registri VERA (solo debug)
`VERA_POST_CDONE_DELAY_US`	1000	attesa fra `CDONE` e il rilascio del reset del C64

2.4 Interfacciamento Bus

Ogni ciclo è qualificato dalle linee `/IO1` e `/IO2` della porta di espansione. Nel C64 le genera il 74LS139 a valle del PLA, che le abilita solo con `AEC` alto, cioè solo per gli accessi della CPU: i fetch del VIC-II, i cui indirizzi compaiono anch'essi sulla porta, non le asseriscono e vengono ignorati. `/IO1` e `/IO2` decodificano già A8–A15, quindi al firmware servono solo A0–A7 come offset nella pagina.

L'adattamento di livello 5 V $\leftrightarrow$ 3.3 V è affidato a cinque **74LVC4245** (lato A a 5 V verso il C64, lato B a 3.3 V), a un invertitore **SN74HCT04** (`U23`) per la direzione del bus dati e a due **SN74LVC00A** (`U24`, `U25`) per l'abilitazione di `U19`; dettaglio in `PIN-MAPPING.md` §6:

- **U19 — Bus dati D0–D7 (GPIO 4–11):** bidirezionale, `DIR` = NOT R/W_ da `U23`. `~OE_ = NAND(PHI2, NAND(R/W, DEV_SEL_N, /IO1))`: acceso in scrittura, nelle letture VERA e nelle letture GeoRAM, spento in tutte le altre letture. L'ESP32 abilita le proprie uscite con `GPIO.enable_w1ts` solo nelle letture della finestra GeoRAM.
- **U4 — A0–A7 (GPIO 12–19)** e **U3 — A8–A15:** ingressi C64→ESP32, con `~OE_` sulla rete `~ABUS_OE` (GPIO 47, sezione 3.1). A8–A15 sono cablati ma non usati dalla decodifica. GPIO 19/20 condividono i pad USB; USB CDC è disabilitato.

- **U6 — PHI2, R/W_, /IO1, /IO2 (GPIO 1, 2, 41, 42):** ingressi C64→scheda; altri tre canali portano `BA`, `/DMA` e `/RESET` a punti di test. GPIO 41 e 42 erano `EXTSEL_N` e `MPD` su RBL-XE.
- **U26, U27 — IRQ della VERA e ARESET (GPIO 37):** uscite scheda→C64 verso le linee condivise `/IRQ` e `/RESET`, attraverso due buffer open-drain SN74LVC1G07; `U16` non pilota più nessuna linea del C64. Una pull-down da 10 kΩ (`R70`) tiene il C64 in reset mentre l'ESP32 si avvia.
- **Connessioni dirette (3.3 V):** `DEV_SEL_N` (GPIO 40) → /CS della VERA e gating di `U19`; `CRESET` (GPIO 38) → reset VERA; `CDONE` (GPIO 39) ← VERA; `ABUS_OE_N` (GPIO 47) → `U3`/`U4`.

PHI2, R/W_, `/IO1` e `/IO2` sono letti con il **Dedicated GPIO input bundle** (`ee.get_gpio_in`, 1 ciclo CPU, canali 0–3); `DEV_SEL_N` è l'unico canale del bundle di uscita (`ee.wr_mask_gpio_out`). I bundle sono creati dal task del bus sul Core 1, che è il core che li usa.

2.5 Ripartizione dei core

Core	Cosa gira	Note
0	`LogTask`, `Serial`	svuota `log_ring` e stampa; qui gira anche l'interrupt della UART e il tick FreeRTOS
1	solo `MonitorTask` (loop del bus)	priorità `configMAX_PRIORITIES - 1`, non cede mai la CPU, **interruzioni mascherate**; watchdog delle interruzioni (MWDT1) disattivato

Nella versione precedente il loop girava sul Core 0 con il tick FreeRTOS attivo: l'interruzione a 1 kHz può far perdere cicli di bus. Ora il Core 1 non esegue nient'altro e non riceve interruzioni; il contatore dei tick resta aggiornato dal Core 0. Il loop passa gli eventi (cambio del latch; accessi ai registri VERA con `VERA_TRACE_REGS=1`) a `LogTask` tramite `log_ring`, un ring buffer *single-producer/single-consumer* senza lock: l'inserimento costa poche istruzioni e non chiama FreeRTOS, il timestamp è aggiunto da `LogTask`. Il cambio di latch viene registrato nella coda di PHI2 e inserito nel ring nel PHI1 successivo.

3 Mapping dei Pin (ESP32-S3FN8, QFN56)

I pin fisici del package QFN56 sono numerati 1–56 (più il pad GND centrale). La tabella riporta i segnali con il numero GPIO logico **e** il numero di pad fisico (Table 2-1 del datasheet ESP32-S3 v2.2).

Nota importante: il MCU *richiede* un **quarzo esterno a 40 MHz** sui pin dedicati **XTAL_P** (pin 54) e **XTAL_N** (pin 53); senza quarzo il chip non si avvia (sezione 3.2).

Segnale	GPIO	Pin QFN56	IO MUX	Dir.	Note
Bus dati D0–D7 — bidirezionale via 74LVC4245 U19 (Bank-0 bit 4–11)					
D0–D7	4–11	9–16	GPIO4–11	I/O	D0 ha un glitch LOW di 60 µs al power-up
Bus indirizzi A0–A7 — input via 74LVC4245 U4 (Bank-0 bit 12–19)					
A0–A2	12–14	17–19	GPIO12–14	IN	
A3	15	21	XTAL_32K_P	IN	pad RTC usato come GPIO; pin 20 = VDD3P3_RTC
A4	16	22	XTAL_32K_N	IN	pad RTC usato come GPIO
A5, A6	17, 18	23, 24	GPIO17, 18	IN	
A7	19	25	USB_D−	IN	USB CDC disabilitato
Bus indirizzi A8–A15 — input via U3, cablati ma non usati dalla decodifica					
A8, A9	20, 21	26, 27	USB_D+, GPIO21	IN	
A10, A11	35, 36	40, 41	GPIO35, 36	IN	
A12, A13	33, 34	38, 39	GPIO33, 34	IN	
A14, A15	45, 46	51, 52	GPIO45, 46	IN	pin di strapping (sezione 3.1)
Controllo — U6 (ingressi) e U27 (uscita open-drain)					
PHI2	1	6	GPIO1	IN	Dedicated GPIO input ch 0; 0.985/1.023 MHz
R/W_	2	7	GPIO2	IN	Dedicated GPIO input ch 1; HIGH = lettura
/IO1	41	47	MTDI	IN	Dedicated GPIO input ch 2; ex `EXTSEL_N`, ex-JTAG
/IO2	42	48	MTMS	IN	Dedicated GPIO input ch 3; ex `MPD`, ex-JTAG
ARESET	37	42	GPIO37	OUT	`/RESET` del C64 via open-drain `U27` (LOW asserito)
Segnali diretti (senza level shifter, 3.3 V)					
DEV_SEL_N	40	45	MTDO	OUT	/CS VERA; Dedicated GPIO output ch 0; ex-JTAG
CRESET	38	43	GPIO38	OUT	/RESET VERA
CDONE	39	44	MTCK	IN	VERA configurata (HIGH = OK); ex-JTAG
ABUS_OE_N	47	37	SPICLK_P	OUT	`~OE` di U3/U4; pull-up `R7`/`R34`
—	3	8	GPIO3	—	non usato; pull-up `R71` 10 kΩ (strapping)
BOOT0	0	5	GPIO0	IN	pulsante BOOT / auto-reset; non usato dal firmware
Debug					
UART TX	43	49	U0TXD	OUT	Serial debug @ 115200 baud
UART RX	44	50	U0RXD	IN	non utilizzato dal firmware

Attenzione: i pin QFN56 30–35 (GPIO 27–32) sono collegati alla flash Quad SPI in-package (variante FN8) e non devono mai essere connessi esternamente; il pin 28 (GPIO 26, `SPICS1`) è il chip select della PSRAM in-package, assente sulla FN8, e resta sconsigliato (datasheet Table 2-14). Resta libero solo GPIO 48 (pin 36). I pin 53 e 54 sono pad analogici dedicati al quarzo da 40 MHz.

3.1 Pin di strapping — comportamento al boot con il C64 collegato

L'ESP32-S3 campiona quattro pin di strapping a ogni reset del chip (accensione, CHIP_PU, programmatore, brown-out), *prima che qualsiasi codice venga eseguito*; i livelli devono essere validi alla salita di CHIP_PU e per 3 ms dopo. Le linee indirizzo della porta di espansione sono sempre pilotate: dal 6510, che continua a pilotarle anche in reset, oppure in PHI1 dal VIC-II e dai pull-up della scheda madre. Il loro livello non può quindi decidere gli strapping di GPIO 45/46.

- **GPIO 0 (pin 5) — boot mode.** Rete `BOOT0` (`R33` 4.7 kΩ verso 3V3, pulsante `SW3`, `C61`, `R54` verso `Q4`); non collegato al C64. CHIP_PU ha `R31` 4.7 kΩ + `C25` 1 µF (4.7 ms) contro 0.47 ms di BOOT0: GPIO 0 è già alto al rilascio del reset. Campionato HIGH → boot SPI dalla flash integrata.
- **GPIO 3 (pin 8) — sorgente JTAG, nessun pull interno.** Non usato dal firmware; pull-up `R71` 10 kΩ verso 3V3, obbligatoria perché il datasheet vieta l'alta impedenza. Con gli eFuse di default il livello è comunque ignorato.

- **GPIO 45 (pin 51) — VDD_SPI** e **GPIO 46 (pin 52) — messaggi ROM / download**, pull-down interni da 45 kΩ, collegati ad A14/A15 tramite `U3`. `~OE` di `U3` e `U4` è la rete `~ABUS_OE`, pilotata da GPIO 47 e tenuta alta da `R7`/`R34` (10 kΩ verso 3V3) mentre l'ESP32 è in reset: i traslatori sono spenti, i pull-down danno GPIO 45 = GPIO 46 = LOW, quindi VDD_SPI = 3.3 V (quella della flash della FN8) e download boot valido anche con il C64 acceso. `setup()` porta GPIO 47 basso molto dopo i 3 ms di hold. Gli eFuse `VDD_SPI_FORCE`/`TIEH` non servono.

3.2 Quarzo oscillatore 40 MHz (pin 53–54)

L'ESP32-S3FN8 richiede un quarzo esterno da **40 MHz** per il riferimento del PLL che alimenta la CPU a 240 MHz, l'interfaccia flash e i divisori UART. I pin sono analogici dedicati, non condivisi con GPIO.

```
XTAL_P (54) --[C1 10pF]-- GND
     |
  [X1 40MHz]    <- Rs 0-33 Ohm in serie (opzionale)
     |
XTAL_N (53) --[C2 10pF]-- GND
```

Componente	Valore	Note
X1 – quarzo	40 MHz, SMD 3225 o 2016	±10 ppm; verificare C_L dal datasheet
C1, C2 – condensatori di carico	10 pF, 0402, ±5 %	$C_{ext} = 2C_L - C_{stray}$; $C_{stray} \approx 3$–5 pF
Rs – resistenza serie	0–33 Ω (opzionale)	Limita il drive dell'oscillatore

Regole di layout PCB: tracce XTAL_P/N più corte possibile (<5 mm); condensatori di carico ai pad del chip; piano di massa solido sotto il quarzo; isolamento dalle tracce PHI2 e D0–D7.

4 Analisi delle Tempistiche

- **Ciclo 6510:** $T_{6510} \approx 1015$ ns (PAL, 0.985 MHz) o ≈ 978 ns (NTSC, 1.023 MHz).
- **Finestra PHI2 alto:** $T_{PHI2\uparrow} \approx 490$ ns. Nella fase PHI1 il bus appartiene al VIC-II: sul C64, a differenza dell'Atari, l'indirizzo della CPU non è disponibile in anticipo e la decodifica può partire solo dopo la salita di PHI2.
- **Vincoli del 6510 a 1 MHz:** dato in lettura richiesto $t_{DSU} = 100$ ns prima della discesa di PHI2; dato in scrittura valido al più $t_{MDS} = 200$ ns dopo la salita e tenuto $t_{HW} = 10$ ns dopo la discesa.
- **Ciclo ESP32 (240 MHz):** $T_{ESP32} \approx 4.17$ ns, quindi

$$\frac{T_{PHI2\uparrow}}{T_{ESP32}} = \frac{490\,\text{ns}}{4.17\,\text{ns}} \approx 117 \text{ cicli ESP32.}$$

4.1 Sequenza di un ciclo

1. **Fronte di salita di PHI2**, letto con `ee.get_gpio_in` (3 cicli per campione); l'istante è annotato con `CCOUNT`.

2. **Qualificazione**: attesa limitata (`IO_QUALIFY_NS`, default 200 ns dalla salita) di `/IO1` o `/IO2` basso, con un solo salto `bnall` sui canali 2–3. Se nessuna delle due si attiva, il ciclo non è della scheda (altro indirizzo o fetch VIC-II) e viene saltato.

3. **Una lettura di `GPIO.in`** (A0–A7): a questo punto `AEC` è alto e l'indirizzo è quello del 6510; R/W_ è già nel canale Dedicated GPIO 1.

4. **`/IO2`**: `c = bus_io2[latch][A0-A7]`, poi `ee.wr_mask_gpio_out`: per $DF00–$DF1F con la VERA selezionata `DEV_SEL_N` va basso (risponde la FPGA).

5. **`/IO1`, finestra GeoRAM in lettura**: l'ESP32 pilota subito D0–D7 con `win[off8]` (`GPIO.out = byte « 4` e `GPIO.enable_w1ts`, due store senza `memw`); `win` è il puntatore alla pagina corrente, tenuto in un registro.

6. **Scritture**: il dato è campionato `BUS_WR_SAMPLE_NS` (300 ns) dopo la salita di PHI2, fra t_{MDS} e la discesa; gli effetti (byte GeoRAM, latch, registri pagina/blocco e ricalcolo di `win`) sono applicati subito, nel resto di PHI2.

7. **Fronte di discesa di PHI2**: bus rilasciato, `DEV_SEL_N` de-asserito, eventuale evento di latch inserito nel ring.

4.2 Percorso critico

I passi 2–5 sono scritti in assembly inline, con tutti gli operandi in registro e le costanti caricate prima dell'attesa. Conteggio dal disassemblato della ELF (`xtensa-esp32s3-elf-objdump`), a partire dal campione di `/IOx` basso:

Fino a	Versione precedente	Versione corren
`DEV_SEL_N` asserito (registri VERA)	≈9 istruzioni + 1 lettura APB	5 istruzioni + 1 lettur
dato GeoRAM pilotato su D0–D7	≈18 istruzioni	≈10 istruzioni + 1 lettura e 2
campionamento di `/IOx`	chiamate C, ≈10 istruzioni	5 istruzioni

La versione precedente teneva la GeoRAM in `.iram0.bss`: una sezione fuori dal layout IRAM di ESP-IDF, che non viene riservata sul bus dati e si sovrappone a `.data`, `.bss` e heap; inoltre la IRAM non è accessibile a byte. Ora la GeoRAM è `.bss` ordinaria in DRAM, come la RAMbo di RBL-XE, e il pilotaggio del bus non usa più la tabella `lut_drive[]`. Con ≈117 cicli disponibili il margine è ampio anche considerando l'attesa di `/IOx`; i tempi reali (latenza delle letture APB, sincronizzatori GPIO, ritardo di `/IOx`) vanno confermati con un analizzatore logico.

La previsione del fronte di discesa usata su RBL-XE non serve: gli effetti delle scritture durano poche decine di cicli e dopo la discesa di PHI2 resta tutto PHI1 (≈490 ns) prima del ciclo successivo.

4.3 Accesso diretto ai registri GPIO

- `GPIO.in` — i 32 GPIO del bank-0 in una lettura: D0–D7, A0–A7 (PHI2, R/W_, `/IO1`, `/IO2` arrivano dai canali Dedicated GPIO).
- `GPIO.out` + `GPIO.enable_w1ts` — pilotaggio del bus dati (`byte « 4`); `GPIO.enable_w1tc` lo rilascia.
- `GPIO.out1_w1ts` — rilascio di `ARESET` all'avvio del loop.

4.4 Diagramma temporale di una lettura GeoRAM

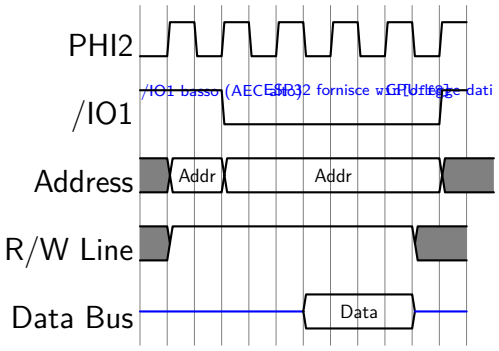


Figura 1: Ciclo di lettura della finestra GeoRAM gestito dall'ESP32.

5 Frammento di Codice Sorgente

Estratti di `bus_decode.h` e `main.cpp` (setup dei bundle e sequenza di reset omessi).

```
#define DEDIC_IN_PHI2         (1u << 0)    /* GPIO 1              */
#define DEDIC_IN_RW           (1u << 1)    /* GPIO 2              */
#define DEDIC_IN_IO1N         (1u << 2)    /* GPIO 41, active LOW */
#define DEDIC_IN_IO2N         (1u << 3)    /* GPIO 42, active LOW */
#define DEDIC_OUT_DEVSELN     (1u << 0)    /* GPIO 40, active LOW */

BUS_INLINE uint32_t bus_io2_ctrl(uint32_t off8, bool selected)
{
    uint32_t c = DEDIC_OUT_CTRL_IDLE | CT_IO2;
    if (selected && off8 < VERA_REG_COUNT)
        c &= ~DEDIC_OUT_DEVSELN;
    if (off8 == CARD_SEL_LATCH || off8 == GEORAM_PAGE_REG || off8 ==
   GEORAM_BLOCK_REG)
        c |= CT_WR_REG;
    return c;
}
```

Listing 1: Canali Dedicated GPIO e tabella di IO2 (bus_decode.h)

```
wait_phi2_high();
const uint32_t t_rise  = cpu_ll_get_cycle_count();
const uint32_t t_limit = t_rise + qualify;
```

```
asm volatile(
    "   movi    %[c], 0x0C                  \n"   /* /IO1, /IO2 in din */
    "   movi    %[t2], 1                    \n"   /* DEDIC_OUT_CTRL_MASK */
    "1: ee.get_gpio_in %[din]               \n"
    "   bnall   %[din], %[c], 2f            \n"   /* /IO1 or /IO2 LOW */
    "   rsr.ccount %[t1]                    \n"
    "   sub     %[t1], %[t1], %[tl]         \n"
    "   bltz    %[t1], 1b                   \n"
    "   movi    %[c], 0                     \n"   /* not ours */
    "   j       6f                          \n"
    "2: l32i    %[lo], %[gpio], %[o_in]     \n"   /* GPIO.in */
    "   extui   %[off], %[lo], 12, 8        \n"   /* A0-A7 */
    "   bbci    %[din], 2, 4f               \n"   /* /IO1 LOW: GeoRAM */
    "   add     %[t1], %[io2], %[off]       \n"
    "   l8ui    %[c], %[t1], 0              \n"   /* io2_cur[A0-A7] */
    "   ee.wr_mask_gpio_out %[c], %[t2]     \n"
    "   j       6f                          \n"
    "4: movi    %[c], %[ct_io1]             \n"
    "   bbci    %[din], 1, 6f               \n"   /* write cycle */
    "   add     %[t1], %[win], %[off]       \n"
    "   l8ui    %[t1], %[t1], 0             \n"   /* win[A0-A7] */
    "   movi    %[t2], %[m_dbus]            \n"
    "   slli    %[t1], %[t1], %[sh]         \n"
    "   s32i    %[t1], %[gpio], %[o_out]    \n"   /* GPIO.out */
    "   s32i    %[t2], %[gpio], %[o_en]     \n"   /* GPIO.enable_w1ts */
    "6:                                     \n"
    /* operandi omessi */);
```

Listing 2: Percorso critico del MonitorTask (Core 1), assembly inline

6 Diagramma di Flusso del Firmware

Il `MonitorTask` gira da solo sul Core 1 a interruzioni mascherate; il Core 0 stampa gli eventi del `log_ring` (sezione 2.5).

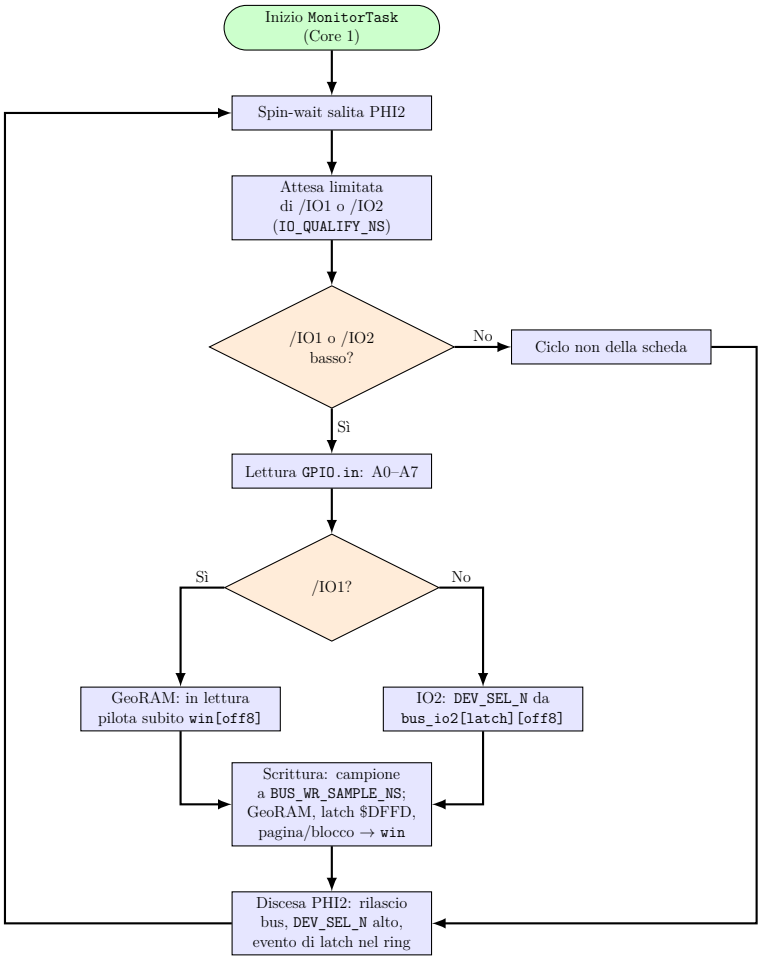


Figura 2: Diagramma di flusso semplificato del `MonitorTask` (ESP32-S3FN8, C64).

7 Verifiche hardware

I punti seguenti derivano dallo schema e dalla netlist (2026-10-26); vanno confermati con l'oscilloscopio. I requisiti della VERA sul bus e il dettaglio delle modifiche sono in `VERA-ATARI-HW-REQUIREMENTS.md`.

- **Ritardo di `/IO1`/`/IO2`.** `IO_QUALIFY_NS` e `BUS_WR_SAMPLE_NS` vanno tarati misurando il ritardo di `/IOx` rispetto a PHI2 (PLA + 74LS139, qualche decina di ns) e il momento in cui il dato in scrittura è stabile.
- **Gating di `U19`** (nello schema). `U24`/`U25` realizzano `NAND(PHI2, NAND(R/W, DEV_SEL_N, /IO1))`: rispetto a RBL-XE l'ingresso `~mEXSEL` è diventato `~mIO1` e `~mMPD` è fisso a 3V3. La GeoRAM è sempre attiva: ogni lettura a $DExx abilita `U19`.
- **Hold del dato in scrittura verso la VERA.** Lo stesso fronte di discesa di PHI2 chiude WR_n e `U19`: verificare che il dato resti stabile dopo la salita di WR_n.
- **`/IRQ` e `/RESET`** passano dai buffer open-drain `U26` e `U27` (nello schema): la scheda tira solo in basso le linee condivise.

- **Linee non usate.** /GAME, /EXROM, /ROML, /ROMH, /NMI e Dot Clock restano scollegate; BA e /DMA arrivano solo a punti di test.
- **PCB** ancora Atari: da rifare con il connettore a pettine della porta di espansione.

Sequenza di reset e configurazione

setup() (Core 1) tiene ARESET e CRESET bassi, avvia LogTask sul Core 0, configura i GPIO, abilita i traslatori degli indirizzi (ABUS_OE_N), rilascia CRESET, attende CDONE (timeout 5 s) e VERA_POST_CDONE_DELAY_US (default 1000 µs) per il POR interno della VERA. Poi crea il MonitorTask sul Core 1, che maschera le interruzioni e solo allora rilascia ARESET. La sequenza vale solo al power-on.

8 Conclusione

Con PHI2 alto per ≈490 ns l'ESP32-S3 a 240 MHz dispone di circa 117 cicli per ciclo di bus, contro i 67 della versione Atari. La qualificazione tramite /IO1 e /IO2 riduce la decodifica alla sola lettura di A0–A7; con il percorso critico in assembly, visto /IOx servono 5 istruzioni e una lettura APB per DEV_SEL_N e circa 10 istruzioni per il dato GeoRAM sul bus.

Il Core 1 è dedicato al bus a interruzioni mascherate e il log passa al Core 0 tramite un ring buffer senza lock. Restano da misurare sulla scheda il ritardo di /IOx, il tempo dal fronte di PHI2 al dato GeoRAM, la latenza degli accessi APB e il comportamento di U19; prima serve il PCB adattato al C64.